

Лекция № 9

Применение потоков. Классификация потоков. Реализация потоков

В многозадачных операционных системах эффективное использование ресурсов процессора, а также параллельная обработка задач становятся ключевыми задачами.

Потоки — это важный механизм для решения этих задач. Они позволяют разделить процесс на несколько параллельно выполняющихся частей, что значительно улучшает производительность и откликаемость системы. В этой лекции мы рассмотрим основные аспекты потоков, их классификацию и методы реализации в современных операционных системах.

1. Применение потоков

Потоки — это основные единицы выполнения внутри процесса. Каждый поток имеет собственный поток исполнения (память, стек и регистры), но все потоки одного процесса разделяют общие ресурсы, такие как память и файловые дескрипторы. Потоки используются для параллельной обработки задач, улучшения отклика приложения и эффективного использования многоядерных процессоров.

Основные области применения потоков:

Многозадачность: Потоки позволяют приложению выполнять несколько операций одновременно. Например, поток может выполнять расчетные операции, в то время как другой поток отвечает за пользовательский интерфейс.

Улучшение отклика: Потоки помогают приложению оставаться отзывчивым, несмотря на выполнение длительных операций. Например, в графических приложениях один поток может рисовать интерфейс, а другой — загружать данные.

Использование многоядерных процессоров: Современные процессоры имеют несколько ядер, и многопоточность позволяет эффективно

использовать все ядра. Например, каждый поток может выполняться на отдельном ядре, что значительно ускоряет выполнение параллельных задач. —

Сетевые и серверные приложения: В серверных приложениях потоки широко используются для обработки множества запросов одновременно. Например, веб-сервер может использовать один поток для обработки каждого входящего запроса.

Асинхронные вычисления: Потоки полезны в ситуациях, когда программа выполняет асинхронные операции, такие как запросы к базам данных или файловым системам. Потоки позволяют не блокировать основное выполнение программы, ожидая завершения этих операций.

2. Классификация потоков

Существует несколько типов потоков, которые можно классифицировать по различным признакам:

2.1. По отношению к процессу

Потоки внутри процесса: Потоки одного процесса разделяют общую память и ресурсы. Эти потоки могут взаимодействовать друг с другом через общую память и обмен сообщениями.

Потоки между процессами: это потоки, которые принадлежат различным процессам. Они могут взаимодействовать друг с другом через механизмы межпроцессного взаимодействия (IPC), такие как очереди сообщений, общая память и т. д.

2.2. По типу исполнения

Пользовательские потоки (user threads): Эти потоки создаются и управляются приложением или библиотеками пользователя. Операционная система не управляет ими напрямую, но предоставляет механизмы для их создания и синхронизации.

Ядровые потоки (kernel threads): Эти потоки управляются ядром операционной системы. Каждый ядровой поток может быть выполнен на отдельном процессорном ядре и может использовать всю мощность многозадачной операционной системы.

2.3. По способу создания

Прямые потоки: Потоки создаются явно в программе с помощью системных вызовов или API, например, с использованием библиотек pthread (POSIX) или Java Thread API.

Непрямые потоки: Эти потоки создаются операционной системой для выполнения системных задач (например, поток, обслуживающий запросы ввода-вывода или системные прерывания).

2.4. По способу управления

Ручное управление потоками: Программист сам управляет созданием, синхронизацией и завершением потоков.

Автоматическое управление потоками: Операционная система или среда выполнения управляет потоками. Например, в таких языках, как Java, потоки могут быть созданы автоматически, и планирование их выполнения управляется системой.

3. Реализация потоков Реализация потоков зависит от операционной системы, языка программирования и среды выполнения.

Рассмотрим два основных подхода к реализации потоков: пользовательские потоки и ядровые потоки.

3.1. Пользовательские потоки

Пользовательские потоки реализуются полностью на уровне приложения, без участия операционной системы. Это означает, что планирование и управление потоками происходит в пользовательском пространстве, а не в ядре операционной системы.

Преимущества:

- Быстрое создание и уничтожение потоков, так как не требуется взаимодействие с ядром.

- Меньшие накладные расходы на управление потоками.

Недостатки:

Пользовательские потоки не могут эффективно использовать многозадачность на многопроцессорных системах, поскольку операционная система видит только один поток для каждого процесса.

Проблемы с блокировкой потока: если один поток в процессе блокируется (например, из-за операции ввода/вывода), все потоки этого процесса также блокируются.

3.2. Ядровые потоки

Ядровые потоки управляются операционной системой. Каждый ядровой поток может быть выполнен на отдельном процессорном ядре, и планирование их выполнения контролируется операционной системой.

Преимущества:

- Полная поддержка многозадачности, эффективное использование многозадачности на многоядерных процессорах.

- Возможность выполнения асинхронных операций без блокировки других потоков.

Недостатки:

- Создание и уничтожение потоков требует обращения к ядру операционной системы, что увеличивает накладные расходы.

- Планирование потоков осуществляется ядром, что может быть менее гибким, чем в случае пользовательских потоков.

3.3. Гибридная модель потоков

Гибридная модель использует и пользовательские, и ядровые потоки. В такой модели потоки в приложении могут быть распределены между

несколькими ядровыми потоками, что позволяет эффективно использовать многозадачность и минимизировать накладные расходы. Например, одна группа пользовательских потоков может быть связана с одним ядровым потоком, и ядровой поток будет управлять всеми пользовательскими потоками.

Контрольные вопросы:

1. Что такое поток в контексте многозадачности операционной системы?
2. Чем отличаются пользовательские потоки от ядровых потоков?
3. Какие преимущества и недостатки имеет использование пользовательских потоков?
4. Что такое гибридная модель потоков и в чем заключается ее преимущество?
5. В чем заключается основное отличие между потоками внутри процесса и потоками между процессами?
6. Как потоки помогают улучшить откликаемость приложений?
7. Какие способы синхронизации потоков существуют и почему они важны?